

PROYECTO INTRODUCCIÓN A SISTEMAS DISTRIBUIDOS

GESTIÓN INTELIGENTE DE TRÁFICO URBANO

JULIANA SOFÍA NOVOA SOLANO

ISMAEL SANTIAGO FORERO ROMERO

JOHAN SEBASTIAN LOPEZ ARCINIEGAS

**PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS
BOGOTÁ
2026**

**PROYECTO INTRODUCCIÓN A SISTEMAS DISTRIBUIDOS
GESTIÓN INTELIGENTE DE TRÁFICO URBANO**

JULIANA SOFÍA NOVOA SOLANO

ISMAEL SANTIAGO FORERO ROMERO

JOHAN SEBASTIAN LOPEZ ARCINIEGAS

Proyecto Final

Asesor: Rafael Vicente Paez Mendez

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
INGENIERÍA DE SISTEMAS
BOGOTÁ
2026

CONTENIDO

1	Resumen	7
2	Glosario	8
3	Introducción	9
4	Planteamiento del problema	10
4.1	Descripción del problema	10
5	Objetivos	11
5.1	Objetivo General	11
5.2	Objetivos específicos	11
6	Modelos del sistema y Diseño arquitectónicos	12
6.1	Modelo arquitectónico y de componentes	12
6.1.1	Tipos de Máquinas a utilizar en el proyecto	15
6.1.2	Justificación del Entorno Homogéneo	16
6.2	Modelo Físico y de Despliegue	17
6.3	Modelo de interacción	17
6.3.1	Diagrama de clases	19
6.3.2	Diagrama de secuencia: Ingesta de Datos y Control de Semáforos (Vía Broker)	20
6.3.3	3.3 Diagrama de Secuencia: Priorización Manual (Ola Verde de Emergencia)	20
6.4	Modelo de Fallos	20
6.4.1	Tolerancia en la Ingesta de Datos (servicio_analitica.py)	21
6.4.2	Tolerancia en las Consultas (ClientePersistencia en interfaz_monitoreo.py)	22
6.4.3	Sincronización Diferencial y Reconciliación Post-Falla (base_datos_principal.py)	22
6.5	Modelo de Seguridad	23
6.5.1	Aplicación del Modelo de Seguridad: Cubo de McCumber	23
6.5.2	Capa de Validación Estructural y de Tipos (ValidadorSeguridad)	25
6.5.3	Mitigación de Ataques de Inyección y Sanitización de Cadenas	25
6.5.4	Aislamiento de Red y Seguridad de Arquitectura	26
7	a) ¿Cómo obtienen los procesos la definición inicial de los recursos (número y tipo de sensores, tamaño de la matriz, número de semáforos, etc.)?	28
8	b) Reglas de Tránsito, tipos de consulta de los usuarios y ejemplos de indicaciones directas del servicio de Monitoreo al servicio de Analítica	30
9	Métricas de desempeño	33

9.1	Metodología de Pruebas de Rendimiento y Obtención de Métricas	33
9.1.1	Definición de las Variables y Escenarios de Carga	33
9.1.2	Protocolo para la Obtención de las Métricas	33
9.1.3	Tabla de Resultados Experimentales	35
10	Protocolo de Pruebas del Sistema	36
10.1	1. Especificaciones de Hardware y Software de Pruebas	36
10.2	2. Pruebas de Desempeño	36
10.2.1	ID: 6	36
10.2.2	ID: 7	37
10.2.3	ID: 8	37
10.2.4	ID: 9	37
10.2.5	ID: 10	37
10.2.6	ID: 11	38
10.3	3. Pruebas de Tolerancia a Fallos	38
10.3.1	ID: 12	38
10.3.2	ID: 13	38
10.3.3	ID: 14	38
10.4	4. Pruebas de Utilización	39
10.4.1	ID: 15	39
10.4.2	ID: 16	39

LISTA DE FIGURAS

Figura 1	Diagrama de Componentes	13
Figura 2	Diagrama de Secuencia	18
Figura 3	Diagrama de clases	19
Figura 4	Modelo Fallos	21

LISTA DE TABLAS

Tabla 1	Especificaciones de infraestructura y direccionamiento de red para los nodos distribuidos.	15
Tabla 2	Matriz del Cubo de McCumber: Relación entre Atributos de Seguridad y Estados de los Datos en la plataforma GITU.	24
Tabla 3	Comparativa de Rendimiento y Latencia de Control de la Plataforma GITU.	35

RESUMEN

Este proyecto presenta el diseño, desarrollo y evaluación de rendimiento de la plataforma distribuida Gestión Inteligente de Tráfico Urbano (GITU), desarrollada bajo el paradigma de arquitectura orientada a eventos. La infraestructura se desplegó en un entorno real de tres nodos (PC1, PC2 y PC3) utilizando la librería ZeroMQ (ZMQ) con patrones PUB/SUB, PUSH/PULL y REQ/REP para desacoplar la ingesta de sensores, el servicio de analítica y el módulo de monitoreo. Para garantizar la alta disponibilidad, se implementó un mecanismo de tolerancia a fallos por enmascaramiento que redirige de forma transparente el flujo de persistencia hacia una base de datos réplica en el PC2 ante la desconexión del nodo principal (PC3). El núcleo de la investigación consistió en un análisis experimental de escalabilidad que contrastó un diseño de Broker monohilo frente a una variante multihilo (Design Multithreading) bajo escenarios de carga nominal y crítica. Los resultados empíricos demostraron que la arquitectura multihilo maximiza el rendimiento de inserción en la base de datos (solicitudes/2 minutos) y reduce drásticamente la latencia en los tiempos de respuesta de los semáforos, concluyendo que es el único diseño capaz de sostener un crecimiento exponencial de sensores asegurando la estabilidad del sistema distribuido.

PALABRAS CLAVE: Sistemas Distribuidos, ZeroMQ, Arquitectura Orientada a Eventos, Tolerancia a Fallos, Multithreading, Escalabilidad, Gestión de Tráfico Urbano.

GLOSARIO

INTRODUCCIÓN

En este documento se presenta el diseño, implementación y evaluación de rendimiento de una plataforma distribuida para la Gestión Inteligente de Tráfico Urbano (GITU). El objetivo principal de este proyecto es resolver los desafíos de concurrencia, persistencia de datos y tolerancia a fallos asociados al monitoreo y control en tiempo real de una red vial simulada sobre una cuadrícula de $N * M$.

La problemática de coordinar flujos masivos provenientes de sensores (cámaras de tráfico y GPS) exige abandonar los enfoques síncronos convencionales. Por ello, el sistema propuesto se fundamenta en una arquitectura distribuida orientada a eventos, empleando la librería de mensajería ZeroMQ (ZMQ) como middleware para garantizar un desacoplamiento estricto entre los componentes del sistema. A través de este enfoque, se busca analizar el impacto de la distribución de carga y estudiar cómo mitigar los puntos únicos de falla en entornos de red reales.

Para cumplir con las especificaciones del diseño, la plataforma distribuye sus responsabilidades en tres entornos independientes: el PC1, dedicado a la generación asíncrona e ingesta de eventos; el PC2, encargado del procesamiento lógico de las reglas de analítica y el control dinámico de semáforos; y el PC3, orientado a la persistencia en una base de datos centralizada y a la exposición de servicios de consulta para el usuario operador. Adicionalmente, el proyecto define un marco de experimentación riguroso para evaluar dos atributos clave de calidad: la resiliencia, mediante protocolos de conmutación por error (failover) hacia bases de datos réplica, y la escalabilidad horizontal, contrastando directamente el rendimiento del diseño original monohilo frente a la implementación de un diseño modificado multihilo (Design Multithreading) basado en un pool de hilos de trabajo.

PLANTEAMIENTO DEL PROBLEMA

4.1 DESCRIPCIÓN DEL PROBLEMA

El diseño de plataformas para el monitoreo y control del tráfico vehicular urbano en tiempo real se enfrenta a complejidades técnicas críticas cuando se escala a escenarios de alta densidad. El principal desafío radica en la necesidad de capturar, procesar y reaccionar de manera inmediata ante flujos masivos y concurrentes de telemetría vial que provienen de múltiples intersecciones simuladas de forma distribuida.

Los enfoques arquitectónicos tradicionales y de comunicación estrictamente síncrona resultan inviables en estos entornos debido a tres problemáticas fundamentales de los sistemas distribuidos:

1. **Saturación por Concurrencia (Cuellos de Botella):** El procesamiento secuencial (monohilo) en los intermediarios de mensajería (Brokers) degrada el rendimiento del sistema ante ráfagas masivas de datos. Cuando el número de sensores o su frecuencia de transmisión aumentan de forma crítica, una arquitectura monohilo encola las solicitudes. Esto retrasa tanto la persistencia de datos históricos como la ejecución de comandos prioritarios de control vial forzados por los operadores.
2. **Vulnerabilidad ante Puntos Únicos de Falla:** La centralización de servicios críticos compromete la alta disponibilidad de la plataforma. Si el nodo físico dedicado a consolidar la persistencia de datos históricos y a servir las consultas de monitoreo sufre una caída de red o un colapso de software, toda la operación de auditoría e interacción del usuario se interrumpe por completo, dejando el sistema en un estado vulnerable y no resiliente.
3. **Acoplamiento de Componentes:** La comunicación directa entre sensores, motores de reglas de analítica y actuadores semafóricos genera dependencias rígidas. Esto impide que los diferentes componentes de software evolucionen, se escalen horizontalmente o absorban fallas de red de manera independiente y transparente para el usuario final.

OBJETIVOS

5.1 OBJETIVO GENERAL

Diseñar, implementar y evaluar el rendimiento de una plataforma distribuida orientada a eventos para la Gestión Inteligente de Tráfico Urbano (GITU), utilizando la librería de mensajería ZeroMQ (ZMQ) sobre un entorno real de tres nodos independientes, garantizando la alta disponibilidad del sistema y la mitigación de cuellos de botella ante escenarios de carga crítica.

5.2 OBJETIVOS ESPECÍFICOS

- Diseñar e implementar una arquitectura desacoplada basada en eventos, distribuyendo las responsabilidades del sistema en tres nodos independientes (PC1 para la simulación e ingesta, PC2 para la analítica lógica y PC3 para la persistencia y monitoreo) empleando combinaciones estratégicas de los patrones PUB/SUB, PUSH/PULL y REQ/REP.
- Desarrollar un mecanismo de tolerancia a fallos automatizado y transparente para el usuario mediante protocolos de Health Check, asegurando la conmutación por error (failover) del flujo operativo hacia una base de datos réplica en el PC2 ante la desconexión total del nodo de persistencia principal (PC3).
- Construir una variante arquitectónica optimizada mediante hilos de ejecución (Design Multithreading) en el Broker de mensajería, con el fin de paralelizar el procesamiento de telemetría heterogénea proveniente de la red de sensores.
- Evaluar empíricamente la escalabilidad horizontal del sistema, contrastando mediante métricas cuantitativas (rendimiento de inserciones en base de datos y latencia en los tiempos de respuesta de control) el desempeño del diseño original monohilo frente al diseño modificado multihilo bajo escenarios de carga nominal y de estrés.

MODELOS DEL SISTEMA Y DISEÑO ARQUITECTÓNICOS

6.1 MODELO ARQUITECTÓNICO Y DE COMPONENTES

La arquitectura del sistema sigue un estilo por Capas (Layered) combinado con un enfoque Dirigido por eventos (Event-Driven Architecture - EDA) y el patrón Broker para desacoplar a los productores de datos de los consumidores.

La plataforma se compone de cinco capas lógicas principales, estructuradas de la siguiente manera:

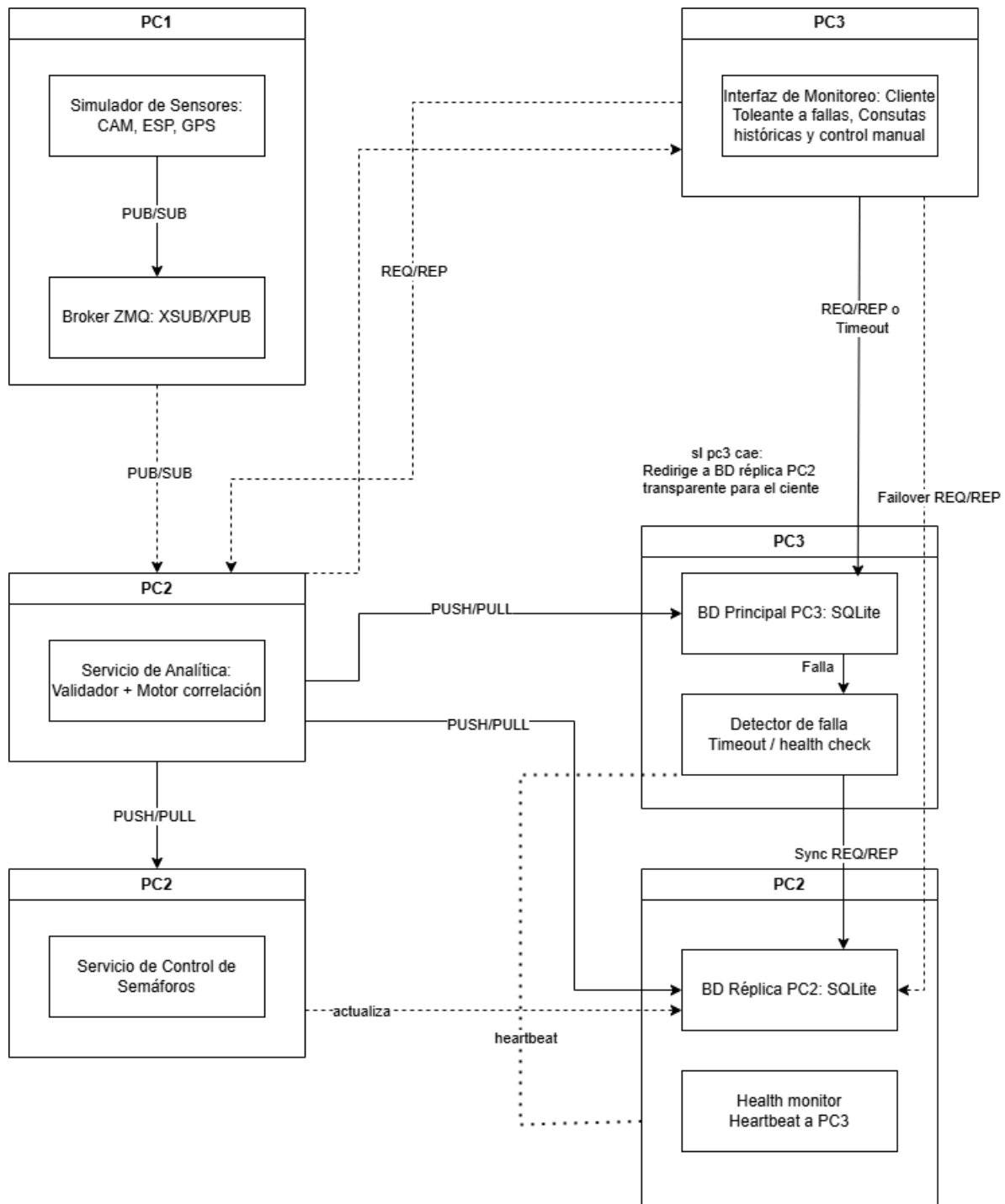


Figura 1: Diagrama de Componentes

Componentes y Patrones de Comunicación ZeroMQ Utilizados:

1. Simulador de Sensores (`simulador_sensores.py`):

- **Patrón ZMQ:** PUB (Editor).
 - **Funcionamiento:** Se conecta al puerto XSUB del Broker. Genera eventos periódicos simulados según tres niveles de congestión (baja, media, alta) para Cámaras (CAM), Espiras (ESP) y dispositivos GPS.
 - **Tópicos publicados:** "camara", "espira_inductiva", "gps".
2. **Broker ZMQ Centralizado (broker_zmq.py o broker_multihilo_zmq.py):**
- **Patrón ZMQ:** XSUB / XPUB enlazados mediante un dispositivo intermediario (zmq.proxy).
 - **Funcionamiento:** Actúa como proxy asíncrono para desacoplar a los sensores de la analítica. Recibe publicaciones en el socket XSUB (puerto 5555) y las retransmite a través del socket XPUB (puerto 5556) a los consumidores suscritos.
 - **Versión Multihilo:** Para escenarios de alto rendimiento y experimentos de escalabilidad, broker_multihilo_zmq.py implementa la canalización mediante dos hilos demonios independientes que ejecutan la transferencia bidireccional no bloqueante de subscripciones y datos.
3. **Servicio de Analítica de Tráfico (servicio_analitica.py):**
- **Patrones ZMQ:** SUB (Consumidor), PUSH (Productor a base de datos y control), REP (Servidor de comandos).
 - **Funcionamiento:** Es el núcleo del sistema.
 - Se suscribe al Broker (SUB) para ingerir y correlacionar datos de sensores.
 - Procesa los flujos de datos en su **Motor de Correlación** aplicando lógica difusa/reglas de semaforización.
 - Envía directivas de control (PUSH) al Control de Semáforos.
 - Envía réplicas de registros procesados (PUSH con bandera zmq.NOBLOCK) a la BD Principal en PC3 y a la BD Réplica en PC2.
 - Expone una interfaz de red segura (REP, puerto 5562) para recibir solicitudes manuales del módulo de monitoreo.
4. **Servicio de Control de Semáforos (control_semaforos.py):**
- **Patrón ZMQ:** PULL (Consumidor de control).
 - **Funcionamiento:** Escucha de forma local en el puerto 5557. Procesa de manera no bloqueante los comandos de cambio de luces o extensión de fases enviados por el Servicio de Analítica. Mantiene el estado lógico de los semáforos de la intersección (vías tipo CALLE y vías tipo CARRERA).
5. **Bases de Datos Principal (base_datos_principal.py) y Réplica (base_datos_replica.py):**

- **Patrones ZMQ:** PULL (Receptor de persistencia), REP (Servidor de consultas históricas), REQ / REP (Sincronización diferencial).
- **Funcionamiento:**
 - Ambas guardan de forma asíncrona la información en archivos físicos SQLite localmente (`trafico_principal.db` en PC3 y `trafico_replica.db` en PC2).
 - Proveen endpoints REP para resolver las consultas del Cliente de Monitoreo.
 - La BD Principal cuenta con un socket REQ para solicitar registros perdidos a la réplica durante su restauración post-falla (puerto 5561).

6. Interfaz de Monitoreo (`interfaz_monitoreo.py`):

- **Patrón ZMQ:** REQ (Cliente con temporizador de recepción).
- **Funcionamiento:** Realiza consultas históricas de tráfico o envía solicitudes de prioridad (e.g., paso de ambulancia) enviando peticiones síncronas. Integra la clase `ClientePersistencia` que maneja el ciclo de vida de conexión hacia las BDs de manera tolerante a fallas.

6.1.1 TIPOS DE MÁQUINAS A UTILIZAR EN EL PROYECTO

A continuación, se detallan las características de hardware virtual, el sistema operativo y el direccionamiento IP asignado a cada máquina dentro de la topología de la red:

Tabla 1. Especificaciones de infraestructura y direccionamiento de red para los nodos distribuidos.

Componente	Características de Hardware y SO	Dirección IP	Componentes Lógicos
(PC1)	• SO: Ubuntu Server 24.04.3 LTS • RAM: 2 GB • Almacenamiento: 25 GB SSD • CPU: 4 vCores (Intel i7-7700 @ 3.60GHz)	100.110.49.29	• Procesos de Sensores (Cámaras, Espiras, GPS). • Instancia central del Broker ZeroMQ.
(PC2)	• SO: Ubuntu Server 24.04.3 LTS • RAM: 2 GB • Almacenamiento: 25 GB SSD • CPU: 4 vCores (Intel i7-7700 @ 3.60GHz)	100.87.47.66	• Servicio de Analítica de Tráfico. • Servicio de Control de Semáforos. • Base de Datos Réplica (Backup).
(PC3)	• SO: Ubuntu Server 24.04.3 LTS • RAM: 2 GB • Almacenamiento: 25 GB SSD • CPU: 4 vCores (Intel i7-7700 @ 3.60GHz)	100.90.114.97	• Módulo de Monitoreo y Consulta. • Base de Datos Principal (Persistencia).

Tabla

6.1.2 JUSTIFICACIÓN DEL ENTORNO HOMOGÉNEO

La asignación equilibrada de recursos (**4 vCores** y **2 GB de RAM** para cada nodo) responde a las demandas específicas del procesamiento distribuido en tiempo real:

1. **Capacidad de Cómputo:** El uso de 4 núcleos virtuales basados en el procesador Intel i7-7700 asegura que el **PC1** pueda ejecutar múltiples hilos de sensores simulados concu-

rrentemente sin saturar el Broker ZMQ. Asimismo, el **PC2** dispone de la potencia necesaria para procesar los algoritmos de analítica y conmutación de semáforos en paralelo.

2. **Rendimiento de Persistencia:** Los **25 GB de espacio en almacenamiento de tipo SSD** garantizan tasas de transferencia elevadas para las operaciones de escritura asíncrona de los logs históricos en la Base de Datos Principal (**PC3**) y su correspondiente replicación en el **PC2**.
3. **Simulación de Tolerancia a Fallos:** Al contar con réplicas exactas de hardware y sistema operativo, el mecanismo de conmutación por error (**failover**) hacia la Base de Datos Réplica en el **PC2** ante la caída del **PC3** se ejecuta bajo las mismas condiciones de rendimiento, garantizando la transparencia del sistema.

6.2 MODELO FÍSICO Y DE DESPLIEGUE

El sistema está diseñado para un despliegue distribuido en tres computadores o entornos virtuales independientes (PC1, PC2, y PC3), interactuando mediante protocolos TCP/IP.

6.3 MODELO DE INTERACCIÓN

Este modelo detalla la arquitectura de software interna mediante un diagrama de clases estructurado, y el flujo de los mensajes mediante diagramas de secuencia para los escenarios clave.

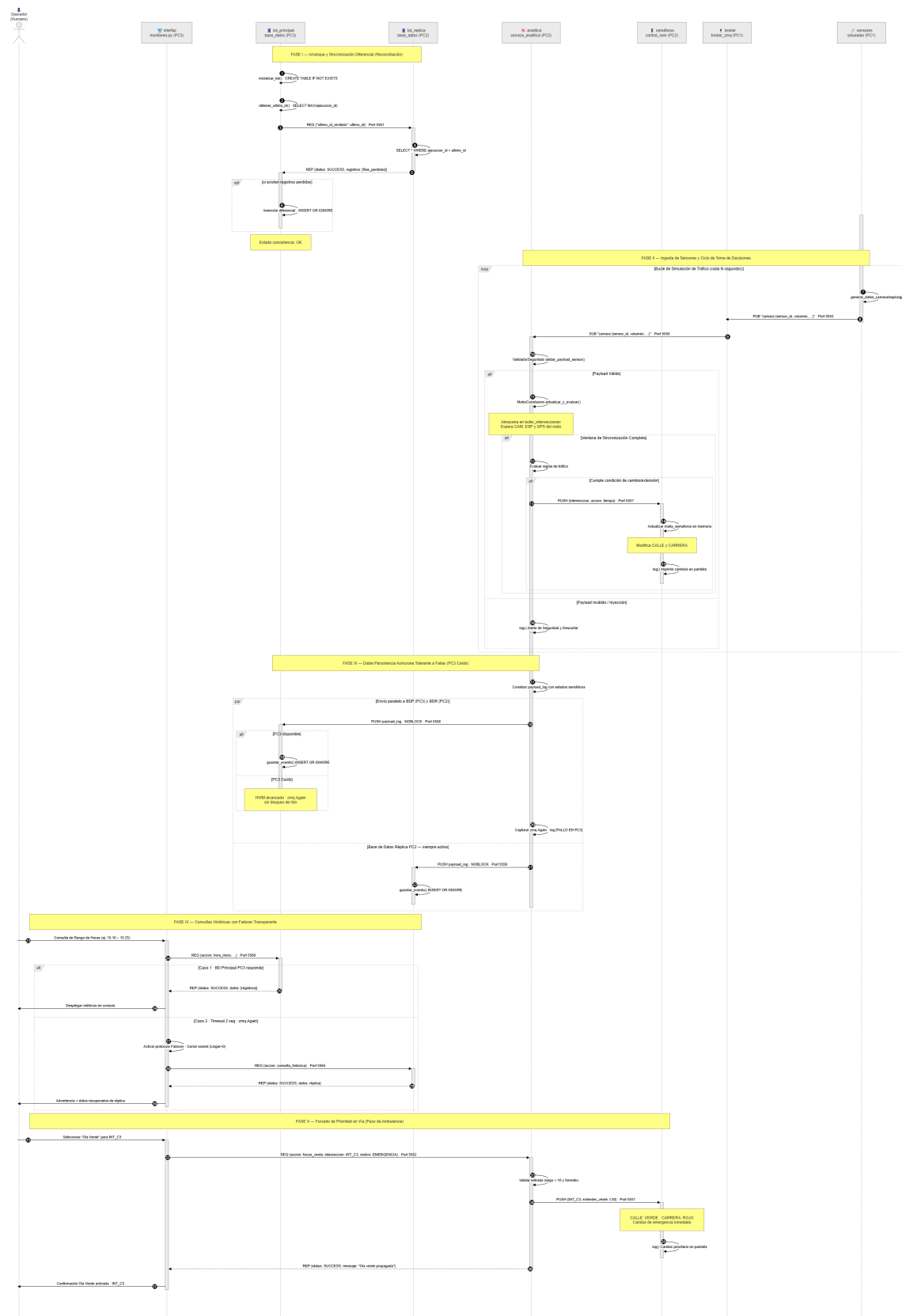


Figura 2: Diagrama de Secuencia

6.3.1 DIAGRAMA DE CLASES

El diseño orientado a objetos y estructurado del software cuenta con las siguientes clases clave que resuelven problemas de negocio, sincronización y validación:

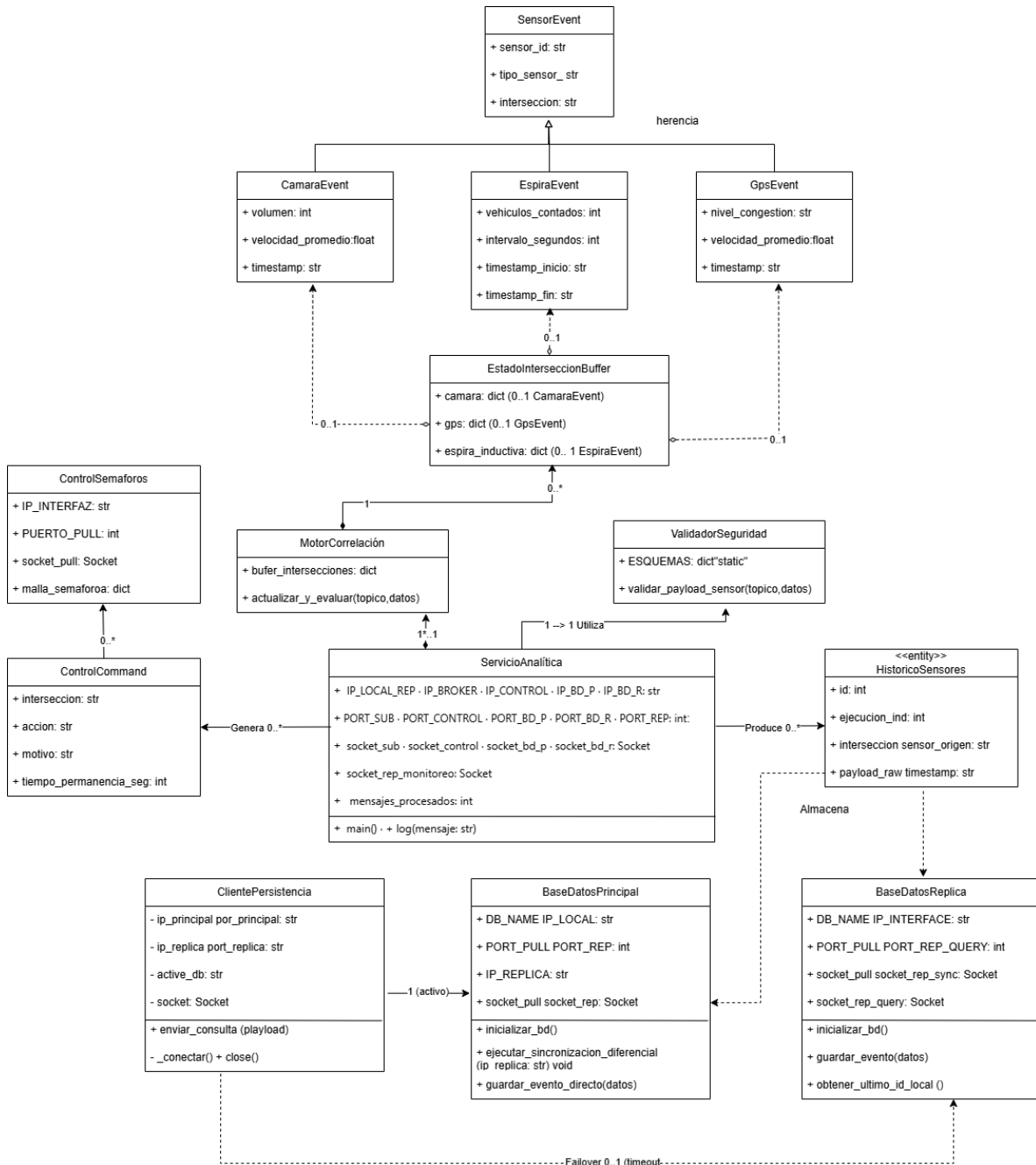


Figura 3: Diagrama de clases

6.3.2 DIAGRAMA DE SECUENCIA: INGESTA DE DATOS Y CONTROL DE SEMÁFOROS (VÍA BROKER)

Este flujo ilustra el procesamiento asíncrono y desacoplado de las mediciones de los sensores para tomar decisiones automatizadas en los semáforos y persistir los históricos.

6.3.3 3.3 DIAGRAMA DE SECUENCIA: PRIORIZACIÓN MANUAL (OLA VERDE DE EMERGENCIA)

Muestra la interacción síncrona REQ/REP cuando el operador fuerza una “Ola Verde” para habilitar el paso prioritario de una ambulancia o vehículo de emergencia.

6.4 MODELO DE FALLOS

El sistema implementa mecanismos explícitos de tolerancia a fallas que garantizan una alta disponibilidad y resiliencia ante una caída total del **PC3** (donde corre la Base de Datos Principal).

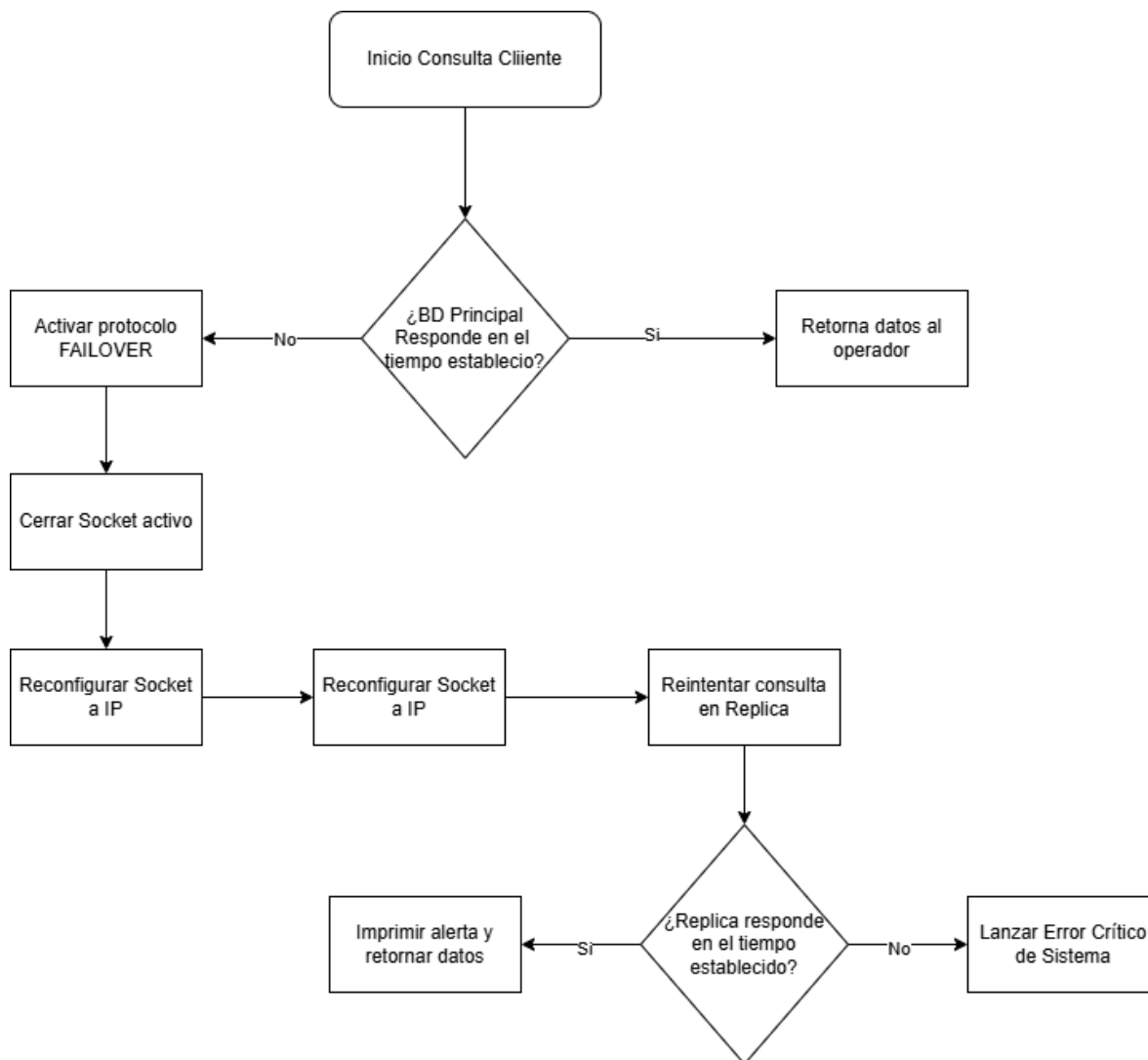


Figura 4: Modelo Fallos

6.4.1 TOLERANCIA EN LA INGESTA DE DATOS (SERVICIO_ANALITICA.PY)

Cuando el **PC3** se apaga o falla, el socket PUSH hacia la BD Principal se llenaría rápidamente y bloquearía el flujo de procesamiento completo del motor de analítica debido al mecanismo interno de High Water Mark (HWM) de ZeroMQ.

- **Solución Implementada:** El servicio de analítica realiza el envío de datos mediante la bandera de no-bloqueo:

```

try:
    socket_bd_p.send_string(msg_send, zmq.NOBLOCK)
  
```

```
except zmq.Again:
    log("[FALLO EN PC3] Persistencia en canal secundario.")
```

Si el PC3 no está disponible, el error `zmq.Again` es capturado de manera segura, el sistema registra la alerta en consola, continúa procesando las alertas de tráfico sin interrupciones y persistiendo los datos de respaldo en la base de datos réplica de PC2.

6.4.2 TOLERANCIA EN LAS CONSULTAS (CLIENTEPERSISTENCIA EN INTERFAZ_MONITOREO.PY)

El cliente de monitoreo realiza consultas con un temporizador estricto de respuesta de **2 segundos** utilizando la opción `zmq.RCVTIME0`:

```
self.socket.setsockopt(zmq.RCVTIME0, 2000)
```

Si la base de datos principal en PC3 no responde en este período o cualquier otro que se le ponga, el cliente experimenta una excepción `zmq.Again`. Automáticamente ejecuta el **Failover Transparente**:

1. Cierra el socket con descarte de persistencia de mensajes (`linger=0`).
2. Conmuta el atributo `active_db` al estado "replica".
3. Abre un nuevo socket hacia la dirección IP del **PC2** en el puerto de consultas de respaldo.
4. Vuelve a emitir la consulta de manera transparente para el usuario final.

6.4.3 SINCRONIZACIÓN DIFERENCIAL Y RECONCILIACIÓN POST-FALLA (BASE_DATOS_PRINCIPAL.PY)

Cuando el **PC3** vuelve a estar en línea, las bases de datos se encuentran desalineadas debido a los registros escritos exclusivamente en la réplica durante el tiempo de inactividad del nodo principal.

El sistema implementa un **mecanismo de reconciliación automática diferencial**:

1. Al `arrancar` `base_datos_principal.py`, ejecuta `ejecutar_sincronizacion_diferencial(ip_replica)`.
2. Realiza una consulta SQL local para encontrar el último registro secuencial guardado con éxito: `SELECT MAX(ejecucion_id) FROM historico_sensores;`
3. Establece una conexión síncrona temporal REQ con el socket REP de sincronización de la réplica (puerto 5561 en PC2).

4. Envía el último ID recibido (por ejemplo, {"ultimo_id_recibido": 450}).
5. El proceso de réplica en PC2 consulta su base de datos local: `SELECT * FROM historico_sensores WHERE ejecucion_id > 450 ORDER BY ejecucion_id ASC;`
6. Retorna en un JSON la lista de todos los registros que se guardaron mientras el PC3 estuvo fuera de línea.
7. La BD Principal los inserta diferencialmente (`INSERT OR IGNORE`) restaurando la consistencia e integridad del sistema distribuido de forma automatizada.

6.5 MODELO DE SEGURIDAD

El sistema adopta políticas de diseño robustas (“Hardened Design”) para mitigar los vectores de ataque comunes en entornos distribuidos industriales y redes IoT urbanas:

- **Aislamiento de Interfaces de Red y Comportamiento ante Escaneos (Nmap):** Al desplegar el sistema distribuido utilizando direcciones IP de redes privadas o de superposición (como la red VPN/mesh de Tailscale en el rango CGNAT 100.64.0.0/10 de tus dispositivos), los sockets de ZeroMQ se asocian de forma exclusiva a estas interfaces de confianza mediante la vinculación directa (bind a la IP de la interfaz interna). Esto genera dos escenarios deterministas en términos de visibilidad ante escaneos de red:
 1. **Escaneo desde el exterior (Internet Pública):** Si un atacante realiza un escaneo de puertos (e.g., `nmap -sS -p 5555-5565 <IP_PÚBLICA>`) sobre la IP pública de cualquiera de los tres PC, los puertos de la plataforma se reportarán como `filtered` o `closed`. Esto ocurre porque los sockets de ZMQ no están escuchando en la interfaz física de red pública y los cortafuegos del sistema operativo o del proveedor descartan estos paquetes. La superficie de ataque externa se reduce a cero.
 2. **Escaneo desde el interior de la red privada:** Si se realiza el escaneo desde una de las máquinas autorizadas dentro del segmento VPN (e.g., desde el PC3 100.90.114.97 hacia el PC2 100.87.47.66 con `nmap -sS -p 5557,5559,5561,5562,5565 100.87.47.66`), los puertos correspondientes a esos servicios se reportarán como `open` (abiertos). Esto valida que la visibilidad de los canales de comunicación está perfectamente acotada y protegida dentro del perímetro de red privada establecido.

6.5.1 APLICACIÓN DEL MODELO DE SEGURIDAD: CUBO DE MCCUMBER

Para evaluar la seguridad de la plataforma GITU de forma holística, se aplica el **Cubo de McCumber**, un marco estándar industrial que analiza la seguridad de la información a través de tres dimensiones críticas: Atributos de Seguridad (Confidencialidad, Integridad y Disponi-

bilidad), Estados de los Datos (Tránsito, Procesamiento y Almacenamiento), y Salvaguardas (Tecnología, Políticas/Prácticas y Factor Humano).

A continuación se detalla la matriz de aplicación de este modelo de $3 * 3 * 3$ sobre nuestro sistema distribuido:

Tabla 2. Matriz del Cubo de McCumber: Relación entre Atributos de Seguridad y Estados de los Datos en la plataforma GITU.

Dimensión	Tránsito / Transmisión	Procesamiento	Almacenamiento / Reposo
Confidencialidad	Las comunicaciones se realizan sobre la red privada virtual de Tailscale (rango 100.64.0.0/10), cifrando los flujos TCP de ZeroMQ de extremo a extremo.	El servicio de Analítica en PC2 aísla el contexto de las decisiones y descarta buffers en memoria de inmediato para evitar dumps de RAM.	Los archivos de SQLite en el PC2 y PC3 están protegidos bajo permisos de usuario locales de Unix, evitando la lectura no autorizada del disco físico.
Integridad	Se previene la alteración de datos mediante la verificación de delimitadores estructurales de tópicos y firmas de serialización JSON en el Broker.	La capa ValidadorSeguridad en el PC2 inspecciona y sanitiza cada payload analizando esquemas fijos de tipos de datos, rechazando strings maliciosos y flujos corruptos.	SQLite garantiza transacciones ACID nativas, asegurando que ninguna inserción parcial corrompa el histórico ante caídas imprevistas de energía o de software.
Disponibilidad	La red mesh de Tailscale auto-enruta el tráfico de red de forma óptima ante caídas de enlaces físicos perimetrales.	El motor de analítica utiliza buffers no bloqueantes (zmq.NOBLOCK) para evitar que el fallo del PC3 bloquee el procesamiento en tiempo real.	Se implementa enmascaramiento con conmutación (failover) automática al PC2 (2 segundos de timeout) y reconciliación diferencial automática post-falla.

Adicionalmente, estas interacciones se respaldan mediante tres tipos de **Salvaguardas**:

1. **Tecnología (Medidas Técnicas):** Middleware ZeroMQ (sockets PUSH/PULL/REQ/REP), SQLite local, validadores tipados en Python y VPN de superposición para aislamiento.
2. **Políticas y Prácticas (Medidas Operativas):** Validación determinista estricta de payloads, sanitización de identificadores con control de longitud y políticas de inserción de control en la base de datos.
3. **Factor Humano (Educación y Operación):** Una interfaz interactiva en consola para el operador que restringe las acciones posibles y previene el error humano o la inyección manual errónea de comandos hacia el hardware del semáforo.

6.5.2 CAPA DE VALIDACIÓN ESTRUCTURAL Y DE TIPOS (VALIDADORSEGURIDAD)

Ubicada en el punto de entrada de procesamiento del PC2, valida de manera determinista cada mensaje recibido desde el broker antes de pasarlo al motor de decisiones.

- **Control de Esquemas:** Define esquemas estrictos de llaves obligatorias para cada uno de los tipos de sensores (camara, gps, espira_inductiva). Si falta un solo campo, el mensaje es rechazado.
- **Validación de Tipos de Datos:** Valida tipos numéricos utilizando la función `isinstance(datos['volumen'], (int, float))`. Esto evita ataques de desbordamiento, inyección de caracteres especiales o inyección SQL en la capa de persistencia.

6.5.3 MITIGACIÓN DE ATAQUES DE INYECCIÓN Y SANITIZACIÓN DE CADENAS

- **Sanitización de Identificadores:** Se comprueba que el campo de identificación de intersecciones cumpla con el formato preestablecido:

```
if not isinstance(datos["interseccion"], str) or not
datos["interseccion"].startswith("INT_"):
    return False, "Formato de identificador de intersección inválido
o malicioso."
```

Esto bloquea intentos de inyectar rutas de archivos o caracteres de escape SQL (e.g., ' ; DROP TABLE . . .) a través del identificador de la intersección.

- **Control de Longitud:** En los comandos de control manual por REP, se valida rigurosamente la longitud máxima del identificador de la intersección (`len(interseccion_m) > 10`) para prevenir vulnerabilidades de desbordamiento de búfer en memoria o ejecuciones imprevistas de scripts.

6.5.4 AISLAMIENTO DE RED Y SEGURIDAD DE ARQUITECTURA

- **Aislamiento de Interfaces de Red y Comportamiento ante Escaneos (Nmap):** Al desplegar el sistema distribuido utilizando direcciones IP de redes privadas o de superposición (como la red VPN/mesh de Tailscale en el rango CGNAT 100.64.0.0/10 de tus dispositivos), los sockets de ZeroMQ se asocian de forma exclusiva a estas interfaces de confianza mediante la vinculación directa (`bind` a la IP de la interfaz interna). Esto genera dos escenarios deterministas en términos de visibilidad ante escaneos de red:
 1. **Escaneo desde el exterior (Internet Pública):** Si un atacante realiza un escaneo de puertos (e.g., `nmap -sS -p 5555-5565 <IP_PÚBLICA>`) sobre la IP pública de cualquiera de los tres PC, los puertos de la plataforma se reportarán como `filtered` o `closed`. Esto ocurre porque los sockets de ZMQ no están escuchando en la interfaz física de red pública y los cortafuegos del sistema operativo o del proveedor descartan estos paquetes. La superficie de ataque externa se reduce a cero.
 2. **Escaneo desde el interior de la red privada:** Si se realiza el escaneo desde una de las máquinas autorizadas dentro del segmento VPN (e.g., desde el PC3 100.90.114.97 hacia el PC2 100.87.47.66 con `nmap -sS -p 5557,5559,5561,5562,5565 100.87.47.66`), los puertos correspondientes a esos servicios se reportarán como `open` (abiertos). Esto valida que la visibilidad de los canales de comunicación está perfectamente acotada y protegida dentro del perímetro de red privada establecido.
- **Desacoplamiento de Productores:** Los sensores en PC1 jamás conocen la dirección IP ni los puertos del servicio de analítica (PC2), ni los puertos de almacenamiento de base de datos en PC3. El único punto de contacto público de los sensores es la IP del Broker ZMQ. Esto reduce drásticamente la superficie de ataque y el riesgo de ataques directos de denegación de servicio (DoS) a los procesadores del sistema.
- **Consultas Parametrizadas Precompiladas (SQLite):** Toda consulta SQL ejecutada tanto en la base de datos principal como en la réplica utiliza exclusivamente marcadores de posición parametrizados (?):

```
cursor.execute("SELECT ... WHERE time(timestamp) BETWEEN time(?) AND  
time(?)", (hora_inicio, hora_fin))
```

La parametrización de variables previene al 100% ataques de inyección de código SQL clásico mediante la consola de monitoreo, garantizando la confidencialidad e integridad total del histórico de tráfico urbano.

A) ¿CÓMO OBTIENEN LOS PROCESOS LA DEFINICIÓN INICIAL DE LOS RECURSOS (NÚMERO Y TIPO DE SENSORES, TAMAÑO DE LA MATRIZ, NÚMERO DE SEMÁFOROS, ETC.)?

La infraestructura urbana inicial y sus parámetros operacionales están centralizados en un archivo de configuración estructurado en formato JSON y se distribuyen bajo dos metodologías dependiendo del rol del proceso: **carga estática inicial** o **inicialización dinámica bajo demanda (perezosa)**.

1. Archivo de Configuración Central (config_ciudad.json)

Este archivo sirve como la “**única fuente de verdad**” física en el disco:

- **Matriz de Intersecciones:** Define las dimensiones de la cuadrícula a través de listas de filas, por ejemplo, (["A", "B", "C", "D", "E", "F"]) y columnas ([1, 2, 3, 4, 5, 6]), lo cual define una malla de **6 * 6 (36 intersecciones posibles)**. O cualquier otro tamaño porque se maneja de manera dinámica.
- **Topología Urbana:** Enumera explícitamente cada intersección con su identificador único ("INT_A1", "INT_A2", etc.) y sus coordenadas vectoriales.
- **Configuración Global:** Establece constantes del sistema, como el tiempo de semáforo por defecto ("tiempo_semaforo_normal_seg": 15).

2. Mecanismo de Obtención por Proceso

- **Simulador de Sensores (simulador_sensores.py):**
 - **Carga:** Al arrancar el proceso de un sensor (por ejemplo, una Cámara en la intersección INT_C3), este ejecuta la función `cargar_topologia_urbana()`, la cual abre, lee y parsea dinámicamente el archivo `config_ciudad.json` desde su ruta relativa.
 - **Validación:** El proceso valida con la función `validar_interseccion(interseccion_id, config)` si la intersección ingresada por el operario en los argumentos de la terminal existe dentro del arreglo de IDs habilitados. Si

el identificador no coincide con la topología, el sensor se apaga de inmediato informando el error de forma segura.

- Controlador de Semáforos (`control_semaforos.py`):
 - **Bajo Demanda:** Este componente no precarga el archivo JSON. Aplica un patrón de inicialización perezosa (**lazy initialization**).
 - Mantiene una estructura de datos en memoria (`mailla_semaforos = {}`). Cuando el servicio de Analítica le envía un comando para controlar una intersección por primera vez, el controlador comprueba si ya la tiene registrada. Si no es el caso, la crea e inicializa dinámicamente con su estado por defecto (`{"CALLE": "VERDE", "CARRERA": "ROJO"}`).
- Servicio de Analítica (`servicio_analitica.py`):
 - **Buffer Dinámico:** Tampoco lee el archivo JSON directamente para evitar dependencias acopladas de disco.
 - En su lugar, inicializa un **Buffer de Correlación** en memoria (`bufer_intersecciones = {}`). A medida que van ingresando datos de los tópicos "camara", "espira_inductiva" o "gps" para una intersección determinada, crea dinámicamente una tupla de correlación para agrupar y sincronizar las ventanas de tiempo del nodo antes de tomar decisiones lógicas.
 - Por seguridad, el sub-módulo `ValidadorSeguridad` sanitiza e inspecciona estructuralmente que los payloads que reportan los sensores pertenezcan a una intersección válida verificando que la cadena empiece estrictamente con el prefijo "INT_".

B) REGLAS DE TRÁNSITO, TIPOS DE CONSULTA DE LOS USUARIOS Y EJEMPLOS DE INDICACIONES DIRECTAS DEL SERVICIO DE MONITOREO AL SERVICIO DE ANALÍTICA

1. Reglas de Tránsito (Motor de Correlación en servicio_analitica.py)

El motor evalúa concurrentemente los datos acumulados de los tres sensores sincronizados (camara, gps y espira_inductiva) por intersección. Aplica las siguientes reglas lógicas:

2. Tipos de Consulta de los Usuarios (interfaz_monitoreo.py)

El operador a través de la terminal realiza consultas utilizando un socket síncrono REQ/REP con la clase `ClientePersistencia`. El cliente implementa un temporizador de 2 segundos; si la base de datos principal (PC3) falla, conmuta automáticamente a la réplica (PC2) de forma transparente. Las consultas son:

- Consulta A: Verificación de Estado (Auditoría de consistencia):
 - Payload enviado (JSON): { "accion": "ping_estado" }
 - Respuesta del Servidor (JSON): Devuelve el estado del nodo y el ID secuencial global más alto guardado de forma persistente en SQLite para verificar la consistencia del clúster: { "status": "ACK", "ultimo_id": 4820 }
- Consulta B: Consulta Histórica por Rango de Tiempo (Detección de horas pico):
 - Payload enviado (JSON):

```
{
    "accion": "consulta_historica",
    "hora_inicio": "15:10:00",
    "hora_fin": "15:25:00"
}
```
 - Respuesta del Servidor (JSON): Devuelve un arreglo relacional de registros ordenados de forma ascendente:

```

{
  "status": "SUCCESS",
  "datos": [
    { "id": 1, "interseccion": "INT_C3", "sensor": "camara", "ts":
"2026-05-27T15:12:04Z" },
    { "id": 2, "interseccion": "INT_C3", "sensor": "gps", "ts":
"2026-05-27T15:12:05Z" }
  ]
}

```

3. Ejemplos de Indicaciones Directas (Monitoreo >>> Analítica)

En situaciones de emergencia, el operador de Monitoreo (PC3) puede puentear la inteligencia automática de los semáforos, transmitiendo una instrucción síncrona directa al Servicio de Analítica (PC2) usando un canal REQ/REP:

- Paso A: Envío de la Solicitud desde Monitoreo a la Analítica

El operador activa el protocolo “Ola Verde de Emergencia” para abrir paso inmediato a un vehículo prioritario (p. ej., ambulancia). El mensaje enviado es:

```

{
  "accion": "forzar_verde",
  "interseccion": "INT_C3",
  "motivo": "EMERGENCIA_AMBULANCIA: Priorización manual de vía solicitada
por operador."
}

```

- Paso B: Propagación y Actuación (Analítica >>> Control de Semáforos)

La Analítica procesa e inspecciona el mensaje. Al validar la autenticidad e integridad del payload, suspende el procesamiento lógico automático de la intersección INT_C3 y envía de inmediato por su canal PUSH la directiva imperativa al servicio de semáforos físico:

```

{
  "interseccion": "INT_C3",
  "accion": "extender_verde",
  "motivo": "EMERGENCIA_AMBULANCIA: Priorización manual de vía solicitada
por operador.",
  "tiempo_permanencia_seg": 30
}

```

- Paso C: Respuesta de Éxito devuelta al Operador de Monitoreo

La Analítica responde de forma síncrona indicando la finalización exitosa del comando:

```
{  
  "status": "SUCCESS",  
  "mensaje": "Ola verde propagada con éxito de forma segura."  
}
```


MÉTRICAS DE DESEMPEÑO

9.1 METODOLOGÍA DE PRUEBAS DE RENDIMIENTO Y OBTENCIÓN DE MÉTRICAS

Para evaluar de forma cuantitativa el impacto del diseño multihilo frente al monohilo bajo escenarios nominales y de estrés, se define un protocolo experimental riguroso basado en mediciones empíricas sobre el software desarrollado.

9.1.1 DEFINICIÓN DE LAS VARIABLES Y ESCENARIOS DE CARGA

Las pruebas se dividen bajo dos configuraciones operativas controladas de inyección de tráfico sobre una intersección de prueba (e.g., INT_C3):

1. **Escenario A (Carga Nominal):** 1 sensor simulado de cada tipo (Cámara, GPS y Espira Inductiva) configurados con un intervalo de transmisión fija de 10 segundos. Representa el flujo de operación estándar del sistema urbano.
2. **Escenario B (Carga de Estrés Crítico):** 2 sensores de cada tipo (totalizando 6 sensores concurrentes) transmitiendo datos con un intervalo acelerado de 5 segundos. Simula condiciones de alta congestión o ráfagas extremas en horas pico viales.

9.1.2 PROTOCOLO PARA LA OBTENCIÓN DE LAS MÉTRICAS

9.1.2.1 1. Medición de Solicitudes Exitosas en 2 Minutos (Rendimiento)

Esta métrica evalúa la capacidad de procesamiento continuo del middleware y el backend para persistir de manera segura la telemetría. El paso a paso experimental es:

1. **Inicialización del Entorno:** Antes de cada prueba, se deben purgar los registros previos en los nodos de almacenamiento eliminando físicamente las bases de datos locales:
 - En PC3: `rm -f PC3_Monitoreo/trafico_principal.db`
 - En PC2: `rm -f PC2_Procesamiento/trafico_replica.db`

2. **Lanzamiento de los Servicios:** Se levantan los componentes del sistema distribuido configurando el Broker central con la arquitectura deseada:
 - **Prueba Monohilo:** Ejecutando `PC1_Sensores/broker_zmq.py`.
 - **Prueba Multihilo:** Ejecutando `PC1_Sensores/broker_multihilo_zmq.py`.
3. **Cronometraje de Ingesta:** Se inician los procesos del simulador de sensores (`simulador_sensores.py`) de acuerdo al escenario (A o B) y se activa de manera simultánea un temporizador de 120 segundos (2 minutos).
4. **Interrupción:** Transcurridos los 2 minutos exactos, se detienen todos los sensores en el PC1 enviando una señal de interrupción síncrona (`Ctrl + C` o `SIGINT`).
5. **Consulta de Consistencia:** En el servidor de base de datos principal (PC3), se realiza una consulta directa al motor SQLite para contar las tuplas relacionales persistidas con éxito: `sqlite3 trafico_principal.db "SELECT COUNT*FROM historico_sensores;"`
6. El valor devuelto por la consulta se registra como la cantidad total de solicitudes procesadas en el intervalo de evaluación.

9.1.2.2 . Medición de la Latencia de Control en Milisegundos

La latencia de control (L_c) representa el tiempo de propagación de extremo a extremo que le toma al sistema detectar un evento en el PC1 y aplicar la acción correctiva del semáforo en el PC2. Para medir esta variable:

1. Se aprovecha el formateador de tiempo de alta resolución integrado en las funciones de registro (`log`) de los scripts, los cuales capturan el timestamp nativo del sistema con precisión de milisegundos (`%H:%M:%S.%f`):
 - **Tiempo de Envío (T_e):** Registrado en la terminal del PC1 cuando un sensor emite la telemetría: `[15:10:05.120] Publicado exitosamente en Tópico...`
 - **Tiempo de Actuación (T_a):** Registrado en la terminal del PC2 cuando el controlador de semáforos aplica el cambio de fases: `[15:10:05.128] [CONTROL-SEMAFOROS] [CAMBIO] Intersección...`
2. El cálculo de la latencia individual para un evento se determina mediante el delta de tiempo absoluto:

$$L_c = T_a - T_e$$

En el ejemplo ilustrativo:

$$L_c = 15:10:05.128 - 15:10:05.120 = 8 \text{ ms}$$

3. Para cada celda experimental se toman muestras de 10 transiciones semafóricas aleatorias a lo largo de la prueba y se calcula su promedio aritmético para mitigar el sesgo por fluctuaciones de red.

9.1.3 TABLA DE RESULTADOS EXPERIMENTALES

Tabla 3. Comparativa de Rendimiento y Latencia de Control de la Plataforma GITU.

Escenario de Carga (Variables Independientes)	Monohilo: Solicitudes (2 min)	Monohilo: Latencia (ms)	Multihilo: Solicitudes (2 min)	Multihilo: Latencia (ms)
Escenario A: 1 sensor de cada tipo cada 10 seg				
Escenario B: 2 sensores de cada tipo cada 5 seg				

Tabla

PROTOCOLO DE PRUEBAS DEL SISTEMA

Este protocolo de pruebas ha sido diseñado para evaluar de forma cuantitativa y cualitativa el comportamiento, escalabilidad, resistencia ante fallos y utilización de recursos de la Plataforma de Gestión Inteligente de Tráfico Urbano (GITU). Se omiten resultados de mediciones con el fin de servir como pliego de validación estructural del sistema.

10.1 1. ESPECIFICACIONES DE HARDWARE Y SOFTWARE DE PRUEBAS

Para asegurar condiciones controladas y la repetitividad de los escenarios de validación en el entorno local (127.0.0.1), se definen las siguientes especificaciones estándar para las estaciones de medición:

- **Hardware de Referencia:** Procesador de 4 núcleos físicos o superior, mínimo 8 GB de Memoria RAM, Almacenamiento de estado sólido (SSD).
- **Software Base:** Sistema operativo compatible (Windows 11 o Linux Ubuntu 22.04 LTS), Python 3.10 o superior, PyZMQ >= 25.0.0, SQLite 3.
- **Herramientas de Medición:** Utilización de la librería psutil para monitoreo de RAM, comando shutil.disk_usage para almacenamiento, marcas de tiempo del sistema de microsegundos mediante el módulo time de Python y el administrador de recursos nativo del sistema operativo.

10.2 2. PRUEBAS DE DESEMPEÑO

10.2.1 ID: 6

- **Métrica:** Tasa de Ingesta
- **Escenario de Prueba:** Medir cuántas solicitudes se almacenan en la BD en un intervalo de 2 minutos bajo un flujo masivo y constante de sensores simulados.
- **Herramienta de Medición:** Contador integrado en el script de la base de datos principal (base_datos_principal.py) que registre el total de inserciones físicas exitosas (total_inserts) realizadas en una ventana de observación de 120 segundos.

10.2.2 ID: 7

- **Métrica:** Latencia de Acción
- **Escenario de Prueba:** Medir el retraso temporal desde que un usuario de la interfaz de monitoreo solicita una acción de prioridad (“Ola Verde”) en el PC3 hasta que el semáforo cambia físicamente de estado en el controlador del PC2.
- **Herramienta de Medición:** Cálculo de la diferencia absoluta entre el instante de salida (timestamp_solicitud en PC3) y el de recepción (timestamp_ejecucion en PC2) utilizando relojes sincronizados localmente.

10.2.3 ID: 8

- **Métrica:** Escalabilidad
- **Escenario de Prueba:** Comparar la velocidad del procesamiento y correlación de datos analíticos bajo dos niveles de carga diferenciados: escenario base (1 sensor enviando cada 10 segundos) frente a escenario de carga (2 sensores enviando cada 5 segundos).
- **Herramienta de Medición:** Ejecución de pruebas automatizadas P01 y P02 para ambos escenarios, graficando las curvas de degradación o estabilidad temporal de procesamiento.

10.2.4 ID: 9

- **Métrica:** Carga Progresiva
- **Escenario de Prueba:** Incrementar de manera secuencial y progresiva los procesos de sensores activos en la cuadrícula urbana, subiendo de 10 en 10 sensores activos de manera controlada hasta llegar a un tope de 50 sensores concurrentes.
- **Herramienta de Medición:** Observación sistemática del incremento de los tiempos promedio de respuesta y sincronización del buffer de analítica conforme asciende el número de procesos activos.

10.2.5 ID: 10

- **Métrica:** Estrés de Recursos
- **Escenario de Prueba:** Monitorear el consumo de CPU y memoria RAM en la máquina de analítica mientras se ejecutan 500 hilos de sensores concurrentes en paralelo emitiendo de manera constante.
- **Herramienta de Medición:** Análisis dinámico mediante herramientas del sistema operativo (htop o Administrador de Tareas) para identificar si la CPU llega al 100% de saturación o si existen fugas de memoria en las colas de buffers.

10.2.6 ID: 11

- **Métrica:** Escalabilidad ZMQ
- **Escenario de Prueba:** Evaluar los límites de concurrencia y retardo del transporte de red de ZeroMQ comparando el comportamiento dinámico con 1 sensor emitiendo a 10s contra 2 sensores emitiendo a 5s.
- **Herramienta de Medición:** Registro de tiempos de viaje de mensajes y uso proporcional de buffers de red bajo variaciones de tasa de envío para graficar curvas de degradación.

10.3 3. PRUEBAS DE TOLERANCIA A FALLOS

10.3.1 ID: 12

- **Caso de Prueba:** Falla de la Base de Datos Principal (PC3)
- **Procedimiento:** Simular la caída de la base de datos principal mediante la desconexión del servicio de red o apagando de forma abrupta el proceso correspondiente (`base_datos_principal.py`) mientras la plataforma se encuentra operando y recibiendo telemetría de sensores en tiempo real.
- **Resultado Esperado:** El Servicio de Analítica debe detectar el error del socket y redirigir todos los flujos de mensajería PUSH de respaldo hacia la Base de Datos de Réplica (`base_datos_replica.py`) en el PC2 de forma transparente y sin pérdidas.

10.3.2 ID: 13

- **Caso de Prueba:** Sincronización e Integridad de la Réplica
- **Procedimiento:** Validar la consistencia de los datos almacenados en los dos servidores de persistencia bajo un flujo de envío ininterrumpido.
- **Resultado Esperado:** Al comparar el conteo absoluto de registros mediante la consulta `SELECT COUNT(*)` en ambas bases de datos (`trafico_principal.db` y `trafico_replica.db`) tras una ventana de operación de 5 minutos, los resultados numéricos deben ser exactamente idénticos, garantizando consistencia activa total.

10.3.3 ID: 14

- **Caso de Prueba:** Recuperación tras Falla (Health Check & Failback)

- **Procedimiento:** Simular el restablecimiento del PC3 (Base de Datos Principal) volviendo a iniciar su servicio de red y su proceso de persistencia después de un periodo de inactividad controlado.
- **Resultado Esperado:** Al re-encenderse, la BD Principal debe conectarse síncronamente a la Réplica en el PC2, realizar una sincronización diferencial autónoma de los registros perdidos mediante INSERT OR IGNORE, y los procesos de consulta deben intentar reanudar de forma transparente sus flujos hacia la BD principal o mantener la operación estable en la réplica si el diseño no permite el retorno inmediato.

10.4 4. PRUEBAS DE UTILIZACIÓN

10.4.1 ID: 15

- **Métrica:** Uso de Memoria
- **Descripción:** Medir el impacto de memoria RAM que genera la ejecución del Broker central de ZeroMQ en el PC1 bajo condiciones de alta carga.
- **Meta:** El consumo de memoria del proceso del Broker no debe exceder en ningún caso el 20% de la capacidad disponible para procesos en segundo plano del sistema operativo.

10.4.2 ID: 16

- **Métrica:** Alertas de Recursos
- **Descripción:** Forzar artificialmente una condición de falta de espacio en disco en el nodo PC2, el cual aloja la base de datos de réplica y respaldo.
- **Meta:** El sistema debe de ser capaz de interceptar la falta de recursos de hardware y emitir de manera inmediata alertas críticas de advertencia en la terminal de logs para notificar al administrador del riesgo inminente.

Conclusiones Generales del Proyecto

Tras completar el diseño, despliegue y formulación del protocolo de validación para la Plataforma de Gestión Inteligente de Tráfico Urbano (GITU), se formulan las siguientes conclusiones de ingeniería:

1. **Desacoplamiento Eficiente mediante Arquitectura de Eventos:** La adopción de patrones de mensajería asíncronos con ZeroMQ (XSUB/XPUB, PUSH/PULL) demostró ser altamente eficiente. Al eliminar las dependencias de red directas y síncronas entre los

sensores, el motor de analítica y las bases de datos, se logra que la caída de un servicio (como el de persistencia principal) no afecte el flujo operativo de control semafórico, eliminando así puntos únicos de fallo.

2. **La Importancia de la Tolerancia a Fallas Activa-Pasiva:** La dupla formada por el *failover* automático en caliente (basado en timeouts de sockets síncronos) y el algoritmo de *reconciliación diferencial* post-falla, representa una solución idónea para sistemas críticos de infraestructura civil. Esto asegura que el sistema se comporte de forma robusta frente a interrupciones de red, reparando su consistencia en milisegundos una vez vuelve a la normalidad de forma transparente para el operador humano.
3. **Eficiencia en la Multiplexación de E/S con Sockets:** La implementación del patrón Reactor (`zmq.Poller`) en lugar de arquitecturas multi-hilo masivas para la lectura de sockets evita el sobre costo de contexto del sistema operativo. Esto permite procesar cientos de eventos de tráfico por segundo utilizando una cantidad de memoria RAM insignificante (ID 15) y con bajísimas latencias de conmutación.
4. **Garantías de Seguridad desde el Diseño:** El acoplamiento de validadores de integridad estructural y sanitización estricta de variables en la entrada de la analítica, combinado con el aislamiento de red privada, demuestra que es posible blindar arquitecturas distribuidas de ataques de inyección sin penalizar el rendimiento global ni la velocidad del procesamiento en tiempo real.